

Pandas II — Cleaning Data, Handling Missing Values, Merging & GroupBy

By the end of today you will be able to take a messy, real-world dataset and clean it completely — handling missing values, removing duplicates, fixing data types, merging tables, and summarising data with groupby. Data cleaning is 70% of a real data scientist's job. Master this and you are already ahead of most beginners.

Dataset for today: Continue using the Titanic CSV from Day 7. Mac users: [source](#) `venv/bin/activate` before running anything.

SECTION 01 Why Data Cleaning Matters So Much

Real-world data is never clean. It comes from humans filling forms, from buggy systems, from multiple sources that do not agree with each other. It has missing values, duplicate rows, wrong data types, inconsistent formatting, and outliers.

What happens if you skip data cleaning and go straight to ML?

Missing values → Most ML algorithms crash with a NaN error.

Duplicates → Your model overfits to repeated rows and gives inflated accuracy.

Wrong data types → A salary stored as text cannot be used in calculations.

Inconsistent text → 'Mumbai', 'mumbai', 'MUMBAI' are treated as three different cities.

Outliers kept in → One wrong salary of Rs. 10 crore pulls predictions off for everyone.

Garbage in = garbage out. A model trained on dirty data gives wrong answers that look correct on the surface. That is the most dangerous kind of wrong.

The professional mindset: cleaning data is not a chore you do before the 'real' work. It IS the real work. A well-cleaned dataset with a simple model beats a dirty dataset with a fancy model every time.

SECTION 02 Handling Missing Values — The Most Common Problem

Missing values in Pandas are represented as NaN (Not a Number). Every dataset you will ever work with has them. You have two choices: drop them or fill them. Knowing when to choose which is the skill.

Step 1 — Find the missing values

Before you fix missing values, you need to know exactly where they are and how many there are.

```
import pandas as pd
import numpy as np

df = pd.read_csv("titanic.csv")

# Total missing values per column
print(df.isnull().sum())

# As a percentage (more meaningful)
missing_pct = (df.isnull().sum() / len(df)) * 100
print(missing_pct.round(1))

# Only show columns that actually have missing values
print(missing_pct[missing_pct > 0].sort_values(ascending=False))

# Check if ANY value in the entire DataFrame is missing
print(df.isnull().any().any())      # True or False

# Total count of all missing values across the whole dataset
print(df.isnull().sum().sum())
```

Output:

```
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

```
Cabin      77.1%
Age        19.9%
Embarked    0.2%
```

```
True
866
```

Step 2 — Drop missing values with dropna()

dropna() removes rows or columns that contain missing values. Use this when the missing data is small in proportion and losing those rows will not significantly reduce your dataset.

```
import pandas as pd
df = pd.read_csv("titanic.csv")

print(f"Original shape: {df.shape}")    # (891, 12)

# Drop ALL rows that have ANY missing value
df_dropped_all = df.dropna()
print(f"After dropna(): {df_dropped_all.shape}")    # (183, 12)
# We lost 708 rows! Because Cabin alone is 77% missing.

# Drop rows missing in SPECIFIC columns only
df_age_clean = df.dropna(subset=["Age"])
print(f"After dropna(Age): {df_age_clean.shape}")    # (714, 12)

# Drop rows missing in EITHER Age OR Embarked
df_two_clean = df.dropna(subset=["Age", "Embarked"])
print(f"After dropna(Age,Embarked): {df_two_clean.shape}")

# Drop columns that are more than 50% missing
threshold = len(df) * 0.5
df_col_clean = df.dropna(axis=1, thresh=threshold)
print(f"After dropping sparse cols: {df_col_clean.shape}")
print(f"Dropped: {set(df.columns) - set(df_col_clean.columns)}")
```

Output:

```
Original shape: (891, 12)
After dropna(): (183, 12)
After dropna(Age): (714, 12)
After dropna(Age,Embarked): (712, 12)
After dropping sparse cols: (891, 11)
Dropped: {'Cabin'}
```

When is it okay to drop rows?

Drop rows → When missing values are random and less than 5% of total rows.

Drop cols → When a column is more than 50-60% missing. It has too little info to be useful.

Never drop blindly. Always check: how much data are you losing?

Dropping 77% of the Cabin column is fine. Dropping 20% of rows due to Age is risky.

You will lose important patterns about younger or older passengers.

Step 3 — Fill missing values with fillna()

`fillna()` replaces missing values with a value you specify. This is called imputation. It keeps all your rows while still making the data usable for ML algorithms.

Strategy 1 — Fill with a fixed value

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Fill Embarked (only 2 missing) with the most common port
most_common_port = df["Embarked"].mode()[0]
print(f"Most common port: {most_common_port}")

df["Embarked"] = df["Embarked"].fillna(most_common_port)
print(f"Embarked missing after fill: {df['Embarked'].isnull().sum()}")
```

Output:

```
Most common port: S
Embarked missing after fill: 0
```

Strategy 2 — Fill with mean or median

For numeric columns, fill with the mean (if data is symmetric) or median (if data has outliers). Age is a good candidate for median because it has a reasonable distribution.

```
import pandas as pd
df = pd.read_csv("titanic.csv")

print(f"Age missing before: {df['Age'].isnull().sum()}") # 177

# Fill with mean
mean_age = df["Age"].mean()
df_mean = df.copy()
df_mean["Age"] = df_mean["Age"].fillna(mean_age)
print(f"Fill with mean ({mean_age:.1f}): {df_mean['Age'].isnull().sum()} missing")

# Fill with median (better for age — less affected by extreme values)
median_age = df["Age"].median()
df["Age"] = df["Age"].fillna(median_age)
print(f"Fill with median ({median_age:.1f}): {df['Age'].isnull().sum()} missing")

# Verify the fix
print(df["Age"].describe())
```

Output:

```
Age missing before: 177
Fill with mean (29.7): 0 missing
Fill with median (28.0): 0 missing
count      891.000000
```

```
mean      29.361582
std       13.019697
min        0.420000
50%       28.000000
max       80.000000
```

Strategy 3 — Fill with forward fill or backward fill

Forward fill (ffill) uses the previous row's value. Backward fill (bfill) uses the next row's value. These are used for time series data where consecutive readings are related.

```
import pandas as pd

# Example: daily temperature readings with some gaps
temps = pd.Series([22.0, None, None, 25.0, None, 27.0])
print("Original:")
print(temps.tolist())

# Forward fill: use last known value
print("\nForward fill (ffill):")
print(temps.ffill().tolist())

# Backward fill: use next known value
print("\nBackward fill (bfill):")
print(temps.bfill().tolist())
```

Output:

```
Original:      [22.0, None, None, 25.0, None, 27.0]
Forward fill:  [22.0, 22.0, 22.0, 25.0, 25.0, 27.0]
Backward fill: [22.0, 25.0, 25.0, 25.0, 27.0, 27.0]
```

Strategy	When to use	Example
dropna()	Missing < 5% of rows, random pattern	Embarked with only 2 missing
fillna(mean)	Numeric column, symmetric, no outliers	Exam scores, temperatures
fillna(median)	Numeric column with outliers	Age, salary, house price
fillna(mode)	Categorical column	City, gender, port of embarkation
ffill / bfill	Time series data, ordered data	Stock prices, sensor readings
Drop the column	Column > 50-60% missing	Cabin in Titanic (77% missing)

Duplicate rows happen when the same record is entered twice — from form re-submissions, from merging two databases, from system errors. They inflate your dataset and make your model think certain patterns are more common than they are.

```
import pandas as pd

# Create a sample dataset WITH duplicates
data = {
    "name" : ["Rahul", "Priya", "Arjun", "Rahul", "Sneha", "Priya"],
    "age" : [22, 21, 23, 22, 20, 21],
    "score" : [85, 92, 78, 85, 95, 92]
}
df = pd.DataFrame(data)
print("Original:")
print(df)

# Check how many duplicates exist
print(f"\nDuplicate rows: {df.duplicated().sum()}")

# See which rows are duplicates
print("\nDuplicate rows:")
print(df[df.duplicated(keep=False)]) # show all copies

# Remove duplicates (keeps first occurrence)
df_clean = df.drop_duplicates()
print(f"\nAfter removing duplicates: {df_clean.shape}")
print(df_clean)

# Remove duplicates based on specific columns only
df_name_unique = df.drop_duplicates(subset=["name"])
print(f"\nUnique names only: {df_name_unique.shape}")
```

Output:

```
Original:
   name  age  score
0  Rahul   22    85
1  Priya   21    92
2  Arjun   23    78
3  Rahul   22    85  <- duplicate of row 0
4  Sneha   20    95
5  Priya   21    92  <- duplicate of row 1
```

```
Duplicate rows: 2
```

```
After removing duplicates: (4, 3)
```

```
   name  age  score
0  Rahul   22    85
1  Priya   21    92
2  Arjun   23    78
4  Sneha   20    95
```

When data is read from a CSV, Pandas makes its best guess about data types. It often gets it wrong. Numbers stored as text, dates stored as strings, categories stored as generic objects. Wrong data types cause silent bugs and failed calculations.

```
import pandas as pd

# Simulate a messy dataset from a real system
messy = pd.DataFrame({
    "age"      : ["22", "21", "23", "20"],      # Numbers as strings
    "salary"   : ["50,000", "65,000", "48,000", "72,000"], # Commas!
    "join_date": ["2022-01-15", "2021-06-01", "2023-03-10", "2020-11-20"],
    "gender"   : ["Male", "Female", "Male", "Female"]
})

print("Before fixing:")
print(messy.dtypes)

# Fix 1: Convert age string to integer
messy["age"] = messy["age"].astype(int)

# Fix 2: Remove commas from salary and convert to integer
messy["salary"] = messy["salary"].str.replace(",", "").astype(int)

# Fix 3: Convert date string to proper datetime
messy["join_date"] = pd.to_datetime(messy["join_date"])

# Fix 4: Convert gender to category (saves memory, better for ML)
messy["gender"] = messy["gender"].astype("category")

print("\nAfter fixing:")
print(messy.dtypes)
print(messy)
```

Output:

Before fixing:

```
age      object
salary   object
join_date object
gender   object
```

After fixing:

```
age      int64
salary   int64
join_date datetime64[ns]
gender    category
```

```
   age  salary  join_date  gender
0   22   50000 2022-01-15    Male
```

```
1    21    65000  2021-06-01  Female
2    23    48000  2023-03-10    Male
3    20    72000  2020-11-20  Female
```

Power of datetime conversion:

Once a column is datetime, you can extract year, month, day, weekday automatically.

```
messy['join_year'] = messy['join_date'].dt.year
messy['join_month'] = messy['join_date'].dt.month
messy['join_weekday'] = messy['join_date'].dt.day_name()
```

These extracted features are often very powerful in ML models.
Day of week and month can predict behaviour patterns.

SECTION 05 String Cleaning — Fixing Text Data

Text columns are almost always messy. Extra spaces, inconsistent capitalisation, typos, leading/trailing whitespace. These make the same value appear as multiple different values. Always clean text before using it.

```
import pandas as pd

df = pd.DataFrame({
    "city"      : ["  Mumbai  ", "DELHI", "bengaluru", "Mumbai", "  Delhi"],
    "product"   : ["Laptop Pro", "laptop pro", "LAPTOP PRO", "Laptop pro",
                  "laptop Pro"]
})

print("Before cleaning:")
print(df["city"].value_counts())

# Remove leading and trailing spaces
df["city"] = df["city"].str.strip()

# Convert to title case (First Letter Capitalised)
df["city"] = df["city"].str.title()

print("\nAfter strip + title:")
print(df["city"].value_counts())

# Standardise product names
df["product"] = df["product"].str.lower().str.strip()

print("\nProduct after lower + strip:")
print(df["product"].value_counts())
```

Output:

```
Before cleaning:
Mumbai      1
DELHI       1
bengaluru   1
Mumbai      1
Delhi       1    <- 5 'different' values, only 3 real cities!

After strip + title:
Mumbai      2
Delhi       2
Bengaluru   1

Product after lower + strip:
laptop pro  5
```

String method	What it does	Example
.str.strip()	Remove spaces from both ends	' Mumbai ' -> 'Mumbai'
.str.lower()	Convert to all lowercase	'DELHI' -> 'delhi'
.str.upper()	Convert to all uppercase	'delhi' -> 'DELHI'
.str.title()	Title case (first letter cap)	'mumbai' -> 'Mumbai'
.str.replace(a,b)	Replace substring a with b	'50,000' -> '50000'
.str.contains(x)	True if string contains x	filter cities containing 'bad'
.str.len()	Length of each string	find entries with short names
.str.split(x)	Split string on delimiter x	split full name into first/last

SECTION 06 Merging DataFrames — Combining Multiple Tables

Merge / Join

What it is: Combining two DataFrames based on a common column (a key). Just like JOIN in SQL. Used when your data is spread across multiple tables and you need to bring it together.

Real-life analogy: Think of a school with two registers. Register A has student names and their roll numbers. Register B has roll numbers and their exam scores. To see each student's score, you combine both registers by matching roll numbers. That is a merge.

In Python: `pd.merge(df1, df2, on='student_id', how='inner')`

The four types of merge — how parameter

how=	What it keeps	When to use
------	---------------	-------------

'inner'	Only rows where key EXISTS in BOTH tables	Most common. Only matched records.
'left'	ALL rows from left table, matched from right	Keep all left rows, fill unmatched right with NaN.
'right'	ALL rows from right table, matched from left	Keep all right rows, fill unmatched left with NaN.
'outer'	ALL rows from BOTH tables	Keep everything, fill mismatches with NaN.

```
import pandas as pd

# Table 1: Student basic info
students = pd.DataFrame({
    "student_id" : [101, 102, 103, 104, 105],
    "name"       : ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"],
    "class"      : ["10A", "10B", "10A", "10B", "10A"]
})

# Table 2: Exam scores (note: student 105 is absent, 106 is not in students)
scores = pd.DataFrame({
    "student_id" : [101, 102, 103, 104, 106],
    "maths"      : [85, 92, 78, 95, 88],
    "science"    : [90, 88, 76, 91, 82]
})

# INNER merge - only students who have BOTH info AND scores
inner = pd.merge(students, scores, on="student_id", how="inner")
print("Inner merge:")
print(inner)

# LEFT merge - all students, NaN for those without scores
left = pd.merge(students, scores, on="student_id", how="left")
print("\nLeft merge:")
print(left)
```

Output:

Inner merge:

	student_id	name	class	maths	science
0	101	Rahul	10A	85	90
1	102	Priya	10B	92	88
2	103	Arjun	10A	78	76
3	104	Sneha	10B	95	91

Left merge:

	student_id	name	class	maths	science	
0	101	Rahul	10A	85.0	90.0	
1	102	Priya	10B	92.0	88.0	
2	103	Arjun	10A	78.0	76.0	
3	104	Sneha	10B	95.0	91.0	
4	105	Mohammed	10A	NaN	NaN	<- absent, no score

Concatenating DataFrames — stacking tables

Sometimes you do not need to match on a key. You just want to stack two tables on top of each other (adding more rows) or side by side (adding more columns). That is `pd.concat()`.

```
import pandas as pd

# Two batches of students from different months
batch_jan = pd.DataFrame({
    "name" : ["Rahul", "Priya"],
    "score" : [85, 92]
})

batch_feb = pd.DataFrame({
    "name" : ["Arjun", "Sneha", "Mohammed"],
    "score" : [78, 95, 88]
})

# Stack vertically (add rows) — axis=0
all_students = pd.concat([batch_jan, batch_feb], axis=0, ignore_index=True)
print("Combined all students:")
print(all_students)
```

Output:

```
Combined all students:
   name  score
0  Rahul    85
1  Priya    92
2  Arjun    78
3  Sneha    95
4 Mohammed    88
```

SECTION 07 GroupBy — Summarising Data by Category

GroupBy

What it is: Split a DataFrame into groups based on values in one or more columns, apply a function to each group, and combine the results. The most powerful summarisation tool in Pandas.

Real-life analogy: Think of a class teacher who has marks for all 200 students across 5 classes. GroupBy is like saying: separate the students by class, then calculate the average score for each class separately. Instead of doing this manually 5 times, groupby does it in one line.

In Python: `df.groupby('class')['score'].mean()`

Basic groupby — one group, one function

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Average fare paid by each passenger class
avg_fare = df.groupby("Pclass")["Fare"].mean()
print("Average Fare by Class:")
print(avg_fare.round(2))

# Survival rate by gender
survival_by_gender = df.groupby("Sex")["Survived"].mean() * 100
print("\nSurvival Rate by Gender (%):")
print(survival_by_gender.round(1))

# Count of passengers by class
count_by_class = df.groupby("Pclass")["PassengerId"].count()
print("\nPassengers per Class:")
print(count_by_class)
```

Output:

Average Fare by Class:

Pclass

1 84.15

2 20.66

3 13.68

Name: Fare, dtype: float64

Survival Rate by Gender (%):

Sex

female 74.2

male 18.9

Name: Survived, dtype: float64

Passengers per Class:

Pclass

1 216

2 184

3 491

Look at what groupby just told us in 3 lines of code: first class passengers paid 6x more than third class, women had a 74% survival rate vs only 19% for men, and two-thirds of passengers were in third class. This is the kind of insight that shapes a whole analysis.

GroupBy with multiple aggregations — agg()

Instead of applying one function, `agg()` lets you apply multiple functions to one or more columns at the same time. This is how you build a complete summary table.

```
import pandas as pd
```

```

df = pd.read_csv("titanic.csv")

# Multiple stats for Fare, grouped by class
fare_stats = df.groupby("Pclass")["Fare"].agg(["mean", "median", "min",
"max", "count"])
print("Fare Statistics by Class:")
print(fare_stats.round(2))

# Multiple columns, multiple functions
summary = df.groupby("Pclass").agg(
    total_passengers = ("PassengerId", "count"),
    survivors        = ("Survived", "sum"),
    survival_rate     = ("Survived", "mean"),
    avg_age           = ("Age", "mean"),
    avg_fare          = ("Fare", "mean")
)
summary["survival_rate"] = (summary["survival_rate"] * 100).round(1)
summary["avg_age"]       = summary["avg_age"].round(1)
summary["avg_fare"]      = summary["avg_fare"].round(2)
print("\nFull Summary by Passenger Class:")
print(summary)

```

Output:

Fare Statistics by Class:

Pclass	mean	median	min	max	count
1	84.15	60.29	0.00	512.33	216
2	20.66	14.25	10.50	73.50	184
3	13.68	8.05	0.00	69.55	491

Full Summary by Passenger Class:

Pclass	total_passengers	survivors	survival_rate	avg_age	avg_fare
1	216	136	63.0	38.2	84.15
2	184	87	47.3	29.9	20.66
3	491	119	24.2	25.1	13.68

Read this output like a story: First class passengers were older (38 avg), paid 84 per fare, and had a 63% survival rate. Third class passengers were youngest (25 avg), paid the least, and had only a 24% survival rate. GroupBy just turned 891 raw rows into a three-row business intelligence table.

GroupBy with multiple group columns

You can group by more than one column at once. This creates a hierarchical index called a MultiIndex.

```

import pandas as pd
df = pd.read_csv("titanic.csv")

```

```
# Survival rate by BOTH class AND gender
cross_tab = df.groupby(["Pclass", "Sex"])["Survived"].agg(
    count      = "count",
    survived    = "sum",
    rate        = "mean"
).round(2)

print(cross_tab)

# Reset index to make it a flat table
flat = cross_tab.reset_index()
print("\nFlat version:")
print(flat)
```

Output:

		count	survived	rate
Pclass	Sex			
1	female	94	91	0.97
	male	122	45	0.37
2	female	76	70	0.92
	male	108	17	0.16
3	female	144	72	0.50
	male	347	47	0.14

This one groupby reveals the entire story of the Titanic: first class women had a 97% survival rate. Third class men had only a 14% survival rate. Women in every class survived at much higher rates than men. This is exactly the kind of insight you would present to a business stakeholder.

SECTION 08 Pivot Tables — A Cleaner Way to Cross-Tabulate

A pivot table is a shortcut for groupby + unstack. It creates a 2D summary table where one column becomes rows, another becomes columns, and the values are aggregated in the cells. You will recognise this from Excel.

```
import pandas as pd
df = pd.read_csv("titanic.csv")

# Pivot table: survival rate by class (rows) and gender (columns)
pivot = df.pivot_table(
    values = "Survived",      # What to calculate
    index  = "Pclass",        # Goes on the rows
    columns = "Sex",          # Goes on the columns
    aggfunc = "mean"          # How to aggregate
).round(3)
```

```

print("Survival rate by Class and Gender:")
print(pivot)

# Pivot table with counts
pivot_count = df.pivot_table(
    values = "PassengerId",
    index = "Pclass",
    columns = "Sex",
    aggfunc = "count"
)
print("\nPassenger counts:")
print(pivot_count)

```

Output:

Survival rate by Class and Gender:

Sex	female	male
Pclass		
1	0.968	0.369
2	0.921	0.157
3	0.500	0.135

Passenger counts:

Sex	female	male
Pclass		
1	94	122
2	76	108
3	144	347

SECTION 09 Full Cleaning Pipeline — Everything Together

This is a complete, professional data cleaning pipeline on the Titanic dataset. This is exactly the kind of code you put in your portfolio and walk through in interviews. Read every line and understand what it does and why.

```

import pandas as pd
import numpy as np

# — Step 1: Load —————
df = pd.read_csv("titanic.csv")
print(f"Step 1 - Loaded: {df.shape}")

# — Step 2: Inspect —————
print("\nStep 2 - Missing values:")
print((df.isnull().sum() / len(df) * 100).round(1))

# — Step 3: Drop useless columns —————
# Cabin is 77% missing. Name and Ticket are identifiers, not features.
df = df.drop(columns=["Cabin", "Name", "Ticket", "PassengerId"])

```

```

print(f"\nStep 3 - After dropping cols: {df.shape}")

# — Step 4: Fill missing Age with median —————
df["Age"] = df["Age"].fillna(df["Age"].median())

# — Step 5: Fill missing Embarked with mode —————
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

# — Step 6: Remove duplicates —————
df = df.drop_duplicates()
print(f"Step 6 - After dedup: {df.shape}")

# — Step 7: Feature engineering —————
df["family_size"] = df["SibSp"] + df["Parch"] + 1
df["is_alone"]     = (df["family_size"] == 1).astype(int)
df["age_group"]    = pd.cut(df["Age"],
                             bins=[0, 12, 18, 60, 100],
                             labels=["Child", "Teen", "Adult", "Senior"])

# — Step 8: Final check —————
print("\nStep 8 - Missing after cleaning:")
print(df.isnull().sum())
print(f"\nFinal shape: {df.shape}")
print("\nSample of cleaned data:")
print(df.head())

# — Step 9: Save the cleaned dataset —————
df.to_csv("titanic_cleaned.csv", index=False)
print("\nCleaned dataset saved to titanic_cleaned.csv")

```

Output:

```

Step 1 - Loaded: (891, 12)

Step 2 - Missing values:
Cabin      77.1%
Age        19.9%
Embarked    0.2%

Step 3 - After dropping cols: (891, 8)
Step 6 - After dedup: (891, 8)

Step 8 - Missing after cleaning:
Survived    0
Pclass      0
Sex          0
Age         0
SibSp       0
Parch       0
Fare        0
Embarked    0

Final shape: (891, 11)
Cleaned dataset saved to titanic_cleaned.csv

```

Notice: we started with 12 columns, 866 missing values, and a messy dataset. We ended with 11 clean columns, 0 missing values, and 3 new engineered features. This cleaned file is what you feed into your ML model in Phase 2.

SECTION 10 Interview Prep — Questions to Answer After Today

Data cleaning questions are asked in almost every junior data science interview. They test whether you actually understand data, not just algorithms.

Q — How do you handle missing values in a dataset? Walk me through your approach.

First I identify the missing values using `isnull().sum()` to see counts and percentage per column. Then I decide the strategy based on what I find. If a column has more than 50-60% missing values, I drop that column entirely because it has too little information. For rows: if the missing data is random and less than 5% of total rows I drop those rows. For numeric columns I fill with the median if data has outliers, or mean if it is symmetric. For categorical columns I fill with the mode, the most frequent value. For time series I use forward fill. The choice always depends on understanding the data domain, not just applying a formula blindly.

Q — What is groupby in Pandas and how have you used it?

GroupBy follows a split-apply-combine pattern. It splits the DataFrame into groups based on a column's values, applies a function to each group independently, and combines the results. I have used it to analyse survival rates by passenger class in the Titanic dataset, to calculate average sales by product category, and to find the distribution of customer segments. The `agg()` method makes it even more powerful because you can apply multiple different functions to multiple columns in a single operation. GroupBy is essentially equivalent to SQL's GROUP BY clause.

Q — What are the different types of joins in Pandas merge? When would you use each?

Pandas merge supports four join types matching SQL joins. Inner join keeps only rows where the key exists in both DataFrames — use when you only want matched records. Left join keeps all rows from the left DataFrame and fills unmatched right values with NaN — use when you want to keep all records from your primary table. Right join is the opposite. Outer join keeps everything from both and fills mismatches with NaN — use when you want the complete picture and can handle NaN values. In practice, inner and left joins cover 90% of real-world cases.

Q — Why is data cleaning considered 70% of a data scientist's job?

Because real-world data comes from imperfect sources: humans filling forms, multiple systems that do not agree, historical records with inconsistent formats, and automated processes that occasionally fail. Before you can build any model, the data needs to be complete (no missing values that crash algorithms), consistent (same city not stored as Mumbai, mumbai, and MUMBAI), correctly typed (salary as a number, not a string), free of duplicates, and meaningful (useless columns removed, useful features engineered). All of this takes time and deep

understanding of the domain. Getting it right is what separates analysts who build reliable models from those who build models that look good on a notebook but fail in production.

SECTION 11 Words to Know

Term	Definition
NaN	Not a Number. How Pandas represents missing values.
isnull()	Returns a DataFrame of True/False showing where values are missing.
dropna()	Remove rows or columns containing missing values.
fillna()	Replace missing values with a specified value or strategy.
Imputation	The process of filling missing values using statistics like mean or median.
ffill	Forward fill. Replace NaN with the previous row's value.
bfill	Backward fill. Replace NaN with the next row's value.
drop_duplicates()	Remove duplicate rows from a DataFrame.
astype()	Convert a column to a different data type.
pd.to_datetime()	Convert a string column to proper datetime type.
.str accessor	Apply string operations to an entire Series: .str.strip(), .str.lower()
pd.merge()	Combine two DataFrames on a common key column. Like SQL JOIN.
pd.concat()	Stack DataFrames vertically (more rows) or horizontally (more columns).
how parameter	Controls which rows to keep in a merge: inner, left, right, outer.
groupby()	Split data by category, apply a function, combine results.
agg()	Apply multiple aggregation functions to grouped data at once.
pivot_table()	Create a 2D summary table. One column as rows, another as columns.
reset_index()	Convert a grouped index back into regular columns.
pd.cut()	Bin a numeric column into categories (e.g. age into Child, Adult, Senior).

SECTION HW Homework — Submit Before Day 9

Create `day8_homework.py` using the Titanic dataset. Push the cleaned CSV and your code to GitHub. From today, every homework submission should include a saved cleaned CSV file.

Exercise 1 — Missing Value Report (Beginner)

Load the Titanic dataset and produce a complete missing value report: column name, count missing, percentage missing. Sort it from highest to lowest percentage. Write one sentence of decision next to each column: drop it, fill with mean, fill with median, fill with mode, or keep as is.

Exercise 2 — Full Cleaning Pipeline (Beginner-Intermediate)

Write a complete cleaning function called `clean_titanic(df)` that: drops the Cabin, Name, Ticket, and PassengerId columns, fills Age with the median, fills Embarked with the mode, removes duplicates, and saves the result to `titanic_cleaned.csv`. Print the shape before and after each step.

Exercise 3 — String Cleaning (Intermediate)

Create a DataFrame of 10 rows with a city column that has intentional messiness: extra spaces, mixed capitalisation, and duplicates caused by inconsistent formatting. Clean it using `.str.strip()` and `.str.title()`. Show `value_counts()` before and after cleaning to prove the fix worked.

Exercise 4 — GroupBy Analysis (Intermediate)

Using the cleaned Titanic dataset, answer these five questions using `groupby` and `agg()`: (1) Average fare paid by each embarkation port. (2) Survival rate by passenger class. (3) Average age of survivors vs non-survivors. (4) Total passengers and survivors grouped by both class and gender. (5) Which embarkation port had the highest survival rate?

Exercise 5 — Merge Practice (Intermediate)

Create two DataFrames manually: one with `product_id`, `product_name`, and `category` for 6 products, and another with `product_id`, `units_sold`, and `revenue` for 5 of those products (one product has no sales yet). Perform an inner merge and a left merge. Print both results and explain in a comment what the difference shows.

Stretch Challenge — Tier A students

Using the Titanic dataset, build a complete EDA report using `groupby` and pivot tables. Answer: (1) Which combination of class, gender, and embarkation port had the best survival odds? (2) Is there a relationship between family size and survival rate? Group passengers into alone, small family (2-4), and large family (5+). (3) Create a pivot table showing average fare by class and embarkation port. (4) Find the top 5 most expensive tickets and look up those passengers by index to see who they were.